

Assignment 5 - Fine Tuning LLAMA Model Using Q-LORA On Banking Data FAQ

```
!pip install -U transformers datasets peft accelerate trl  
sentencepiece bitsandbytes
```

```
Requirement already satisfied: transformers in  
/usr/local/lib/python3.13/dist-packages (5.19.0)  
Requirement already satisfied: datasets in  
/usr/local/lib/python3.13/dist-packages (5.1.0)  
Requirement already satisfied: peft in /usr/local/lib/python3.13/dist-  
packages (0.21.2)  
Requirement already satisfied: accelerate in  
/usr/local/lib/python3.13/dist-packages (1.15.0)  
Requirement already satisfied: trl in /usr/local/lib/python3.13/dist-  
packages (1.15.0)  
Requirement already satisfied: sentencepiece in  
/usr/local/lib/python3.13/dist-packages (0.2.2)  
Requirement already satisfied: bitsandbytes in  
/usr/local/lib/python3.13/dist-packages (0.50.2)  
Requirement already satisfied: huggingface-hub<3.0,>=1.31.0 in  
/usr/local/lib/python3.13/dist-packages (from transformers) (1.33.0)  
Requirement already satisfied: numpy>=1.17 in  
/usr/local/lib/python3.13/dist-packages (from transformers) (2.1.3)  
Requirement already satisfied: packaging>=20.0 in  
/usr/local/lib/python3.13/dist-packages (from transformers) (26.3)  
Requirement already satisfied: pyyaml>=5.1 in  
/usr/local/lib/python3.13/dist-packages (from transformers) (6.0.3)  
Requirement already satisfied: regex>=2025.10.22 in  
/usr/local/lib/python3.13/dist-packages (from transformers)  
(2025.11.3)  
Requirement already satisfied: tokenizers<0.24.0,>=0.23.1 in  
/usr/local/lib/python3.13/dist-packages (from transformers) (0.23.2)  
Requirement already satisfied: typer in  
/usr/local/lib/python3.13/dist-packages (from transformers) (0.27.2)  
Requirement already satisfied: safetensors>=0.8.0 in  
/usr/local/lib/python3.13/dist-packages (from transformers) (0.8.0)  
Requirement already satisfied: tqdm>=4.60 in  
/usr/local/lib/python3.13/dist-packages (from transformers) (4.67.3)  
Requirement already satisfied: filelock in  
/usr/local/lib/python3.13/dist-packages (from datasets) (4.0.7)  
Requirement already satisfied: pyarrow>=24.0.0 in  
/usr/local/lib/python3.13/dist-packages (from datasets) (26.0.0)  
Requirement already satisfied: dill<0.4.2,>=0.3.0 in  
/usr/local/lib/python3.13/dist-packages (from datasets) (0.4.1)  
Requirement already satisfied: pandas in  
/usr/local/lib/python3.13/dist-packages (from datasets) (2.2.3)  
Requirement already satisfied: xxhash in  
/usr/local/lib/python3.13/dist-packages (from datasets) (4.0.1)  
Requirement already satisfied: multiprocessing<0.70.20 in
```

/usr/local/lib/python3.13/dist-packages (from datasets) (0.70.19)
Requirement already satisfied: fsspec<=2026.7.0,>=2023.1.0 in
/usr/local/lib/python3.13/dist-packages (from
fsspec[http]<=2026.7.0,>=2023.1.0->datasets) (2025.12.0)
Requirement already satisfied: psutil in
/usr/local/lib/python3.13/dist-packages (from peft) (5.9.5)
Requirement already satisfied: torch>=1.13.0 in
/usr/local/lib/python3.13/dist-packages (from peft) (2.11.0+cu130)
Requirement already satisfied: jinja2 in
/usr/local/lib/python3.13/dist-packages (from trl) (3.1.6)
Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in
/usr/local/lib/python3.13/dist-packages (from
fsspec[http]<=2026.7.0,>=2023.1.0->datasets) (3.14.3)
Requirement already satisfied: click<9.0.0,>=8.4.2 in
/usr/local/lib/python3.13/dist-packages (from huggingface-
hub<3.0,>=1.31.0->transformers) (8.5.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.6.0 in
/usr/local/lib/python3.13/dist-packages (from huggingface-
hub<3.0,>=1.31.0->transformers) (1.6.0)
Requirement already satisfied: httpx<1,>=0.23.0 in
/usr/local/lib/python3.13/dist-packages (from huggingface-
hub<3.0,>=1.31.0->transformers) (0.28.1)
Requirement already satisfied: typing-extensions>=4.1.0 in
/usr/local/lib/python3.13/dist-packages (from huggingface-
hub<3.0,>=1.31.0->transformers) (4.16.0)
Requirement already satisfied: setuptools<82 in
/usr/local/lib/python3.13/dist-packages (from torch>=1.13.0->peft)
(80.10.2)
Requirement already satisfied: sympy>=1.13.3 in
/usr/local/lib/python3.13/dist-packages (from torch>=1.13.0->peft)
(1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.13/dist-packages (from torch>=1.13.0->peft)
(3.7)
Requirement already satisfied: cuda-toolkit==13.0.2 in
/usr/local/lib/python3.13/dist-packages (from cuda-
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenv,
tlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch>=1.13.0-
>peft) (13.0.2)
Requirement already satisfied: cuda-bindings<14,>=13.0.3 in
/usr/local/lib/python3.13/dist-packages (from torch>=1.13.0->peft)
(13.0.3)
Requirement already satisfied: nvidia-cudnn-cu13==9.19.0.56 in
/usr/local/lib/python3.13/dist-packages (from torch>=1.13.0->peft)
(9.19.0.56)
Requirement already satisfied: nvidia-cusparselt-cu13==0.8.0 in
/usr/local/lib/python3.13/dist-packages (from torch>=1.13.0->peft)
(0.8.0)
Requirement already satisfied: nvidia-nccl-cu13==2.28.9 in

```
/usr/local/lib/python3.13/dist-packages (from torch>=1.13.0->peft)
(2.28.9)
Requirement already satisfied: nvidia-nvshmem-cu13==3.4.5 in
/usr/local/lib/python3.13/dist-packages (from torch>=1.13.0->peft)
(3.4.5)
Requirement already satisfied: triton==3.6.0 in
/usr/local/lib/python3.13/dist-packages (from torch>=1.13.0->peft)
(3.6.0)
Requirement already satisfied: nvidia-cublas==13.1.0.3.* in
/usr/local/lib/python3.13/dist-packages (from cuda-
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars
e,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch>=1.13.0-
>peft) (13.1.0.3)
Requirement already satisfied: nvidia-cuda-runtime==13.0.96.* in
/usr/local/lib/python3.13/dist-packages (from cuda-
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars
e,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch>=1.13.0-
>peft) (13.0.96)
Requirement already satisfied: nvidia-cufft==12.0.0.61.* in
/usr/local/lib/python3.13/dist-packages (from cuda-
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars
e,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch>=1.13.0-
>peft) (12.0.0.61)
Requirement already satisfied: nvidia-cufile==1.15.1.6.* in
/usr/local/lib/python3.13/dist-packages (from cuda-
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars
e,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch>=1.13.0-
>peft) (1.15.1.6)
Requirement already satisfied: nvidia-cuda-cupti==13.0.85.* in
/usr/local/lib/python3.13/dist-packages (from cuda-
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars
e,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch>=1.13.0-
>peft) (13.0.85)
Requirement already satisfied: nvidia-curand==10.4.0.35.* in
/usr/local/lib/python3.13/dist-packages (from cuda-
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars
e,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch>=1.13.0-
>peft) (10.4.0.35)
Requirement already satisfied: nvidia-cusolver==12.0.4.66.* in
/usr/local/lib/python3.13/dist-packages (from cuda-
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars
e,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch>=1.13.0-
>peft) (12.0.4.66)
Requirement already satisfied: nvidia-cuspars
e==12.6.3.3.* in
/usr/local/lib/python3.13/dist-packages (from cuda-
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars
e,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch>=1.13.0-
>peft) (12.6.3.3)
Requirement already satisfied: nvidia-nvjitlink==13.0.88.* in
```

```
/usr/local/lib/python3.13/dist-packages (from cuda-  
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars  
e,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch>=1.13.0-  
>peft) (13.0.88)  
Requirement already satisfied: nvidia-cuda-nvrtc==13.0.88.* in  
/usr/local/lib/python3.13/dist-packages (from cuda-  
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars  
e,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch>=1.13.0-  
>peft) (13.0.88)  
Requirement already satisfied: nvidia-nvtx==13.0.85.* in  
/usr/local/lib/python3.13/dist-packages (from cuda-  
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cuspars  
e,nvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch>=1.13.0-  
>peft) (13.0.85)  
Requirement already satisfied: MarkupSafe>=2.0 in  
/usr/local/lib/python3.13/dist-packages (from jinja2->trl) (3.0.3)  
Requirement already satisfied: python-dateutil>=2.8.2 in  
/usr/local/lib/python3.13/dist-packages (from pandas->datasets)  
(2.9.0.post0)  
Requirement already satisfied: pytz>=2020.1 in  
/usr/local/lib/python3.13/dist-packages (from pandas->datasets)  
(2025.2)  
Requirement already satisfied: tzdata>=2022.7 in  
/usr/local/lib/python3.13/dist-packages (from pandas->datasets)  
(2026.4)  
Requirement already satisfied: shellingham>=1.3.0 in  
/usr/local/lib/python3.13/dist-packages (from typer->transformers)  
(1.5.4)  
Requirement already satisfied: rich>=13.8.0 in  
/usr/local/lib/python3.13/dist-packages (from typer->transformers)  
(13.9.4)  
Requirement already satisfied: annotated-doc>=0.0.2 in  
/usr/local/lib/python3.13/dist-packages (from typer->transformers)  
(0.0.5)  
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in  
/usr/local/lib/python3.13/dist-packages (from aiohttp!=4.0.0a0,!  
=4.0.0a1->fsspec[http]<=2026.7.0,>=2023.1.0->datasets) (2.7.1)  
Requirement already satisfied: aiosignal>=1.4.0 in  
/usr/local/lib/python3.13/dist-packages (from aiohttp!=4.0.0a0,!  
=4.0.0a1->fsspec[http]<=2026.7.0,>=2023.1.0->datasets) (1.4.0)  
Requirement already satisfied: attrs>=17.3.0 in  
/usr/local/lib/python3.13/dist-packages (from aiohttp!=4.0.0a0,!  
=4.0.0a1->fsspec[http]<=2026.7.0,>=2023.1.0->datasets) (26.1.0)  
Requirement already satisfied: frozenlist>=1.1.1 in  
/usr/local/lib/python3.13/dist-packages (from aiohttp!=4.0.0a0,!  
=4.0.0a1->fsspec[http]<=2026.7.0,>=2023.1.0->datasets) (1.8.0)  
Requirement already satisfied: multidict<7.0,>=4.5 in  
/usr/local/lib/python3.13/dist-packages (from aiohttp!=4.0.0a0,!  
=4.0.0a1->fsspec[http]<=2026.7.0,>=2023.1.0->datasets) (6.9.1)
```

```

Requirement already satisfied: propcache>=0.2.0 in
/usr/local/lib/python3.13/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2026.7.0,>=2023.1.0->datasets) (0.5.4)
Requirement already satisfied: yarll<2.0,>=1.17.0 in
/usr/local/lib/python3.13/dist-packages (from aiohttp!=4.0.0a0,!
=4.0.0a1->fsspec[http]<=2026.7.0,>=2023.1.0->datasets) (1.25.1)
Requirement already satisfied: cuda-pathfinder~=1.1 in
/usr/local/lib/python3.13/dist-packages (from cuda-
bindings<14,>=13.0.3->torch>=1.13.0->peft) (1.8.2)
Requirement already satisfied: anyio in
/usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.23.0-
>huggingface-hub<3.0,>=1.31.0->transformers) (4.15.1)
Requirement already satisfied: certifi in
/usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.23.0-
>huggingface-hub<3.0,>=1.31.0->transformers) (2026.7.22)
Requirement already satisfied: httpcore==1.* in
/usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.23.0-
>huggingface-hub<3.0,>=1.31.0->transformers) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.13/dist-
packages (from httpx<1,>=0.23.0->huggingface-hub<3.0,>=1.31.0-
>transformers) (3.20)
Requirement already satisfied: h11>=0.16 in
/usr/local/lib/python3.13/dist-packages (from httpcore==1.*-
>httpx<1,>=0.23.0->huggingface-hub<3.0,>=1.31.0->transformers)
(0.16.0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.2-
>pandas->datasets) (1.17.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.13/dist-packages (from rich>=13.8.0->typer-
>transformers) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.13/dist-packages (from rich>=13.8.0->typer-
>transformers) (2.21.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.13/dist-packages (from sympy>=1.13.3-
>torch>=1.13.0->peft) (1.3.0)
Requirement already satisfied: mdurl~=0.1 in
/usr/local/lib/python3.13/dist-packages (from markdown-it-py>=2.2.0-
>rich>=13.8.0->typer->transformers) (0.1.2)

```

1. Import Packages

```

import torch
import transformers
import datasets
import peft
import trl

print("PyTorch:", torch.__version__)

```

```

print("Transformers:", transformers.__version__)
print("Datasets:", datasets.__version__)
print("PEFT:", peft.__version__)
print("Accelerate:", transformers.__version__)
print("TRL:", trl.__version__)

PyTorch: 2.11.0+cu130
Transformers: 5.19.0
Datasets: 5.1.0
PEFT: 0.21.2
Accelerate: 5.19.0
TRL: 1.15.0

import torch

device = torch.device("cpu")

print("Device:", device)
print("CUDA available:", torch.cuda.is_available())
print("CPU cores:", torch.get_num_threads())

Device: cpu
CUDA available: True
CPU cores: 1

model_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"

from datasets import load_dataset

url =
"https://raw.githubusercontent.com/Shivam859526/Banking-Data/refs/heads/main/banking\_faqs\_questions\_responses\_only.json"

dataset = load_dataset(
    "json",
    data_files=url,
    split="train"
)

# Create the text column while preparing the dataset
dataset = dataset.map(
    lambda x: {
        "text": f"### Question:\n{x['question']}\n\n### Answer:\n{x['response']}"
    },
    remove_columns=["question", "response"]
)

print(dataset)
print(dataset[0]["text"])

```

```
{"model_id": "51c44eeca3064e239db05279bf1d10e0", "version_major": 2, "version_minor": 0}
```

```
Dataset({  
  features: ['text'],  
  num_rows: 100  
})
```

Question:

How can I open a bank account?

Answer:

You can usually open an account online or at a branch. You will need to complete the application, provide required identity and address documents, and complete any verification required by the bank.

1. Create Train And Test Split

```
dataset_split = dataset.train_test_split(test_size=0.1, seed=2)  
  
train_dataset = dataset_split["train"]  
test_dataset = dataset_split["test"]  
  
print(f"Train dataset size: {len(train_dataset)}")  
print(f"Test dataset size: {len(test_dataset)}")
```

```
Train dataset size: 90  
Test dataset size: 10
```

1. Load Tokenizer

```
from transformers import AutoTokenizer  
  
tokenizer = AutoTokenizer.from_pretrained(model_name)  
tokenizer.pad_token = tokenizer.eos_token  
  
print("Vocabulary size:", tokenizer.vocab_size)  
print("EOS tokens:", tokenizer.eos_token)  
print("PAD tokens:", tokenizer.pad_token)
```

```
Vocabulary size: 32000  
EOS tokens: </s>  
PAD tokens: </s>
```

1. Quantization

```
from transformers import BitsAndBytesConfig  
  
bnb_config = BitsAndBytesConfig(  
    load_in_4bit = True,  
    bnb_4bit_use_double_quant = True,  
    bnb_4bit_quant_type = "nf4",
```

```

        bnb_4bit_compute_dtype = torch.float32
    )

print("BNB config:", bnb_config)
BNB config: BitsAndBytesConfig {
  "_load_in_4bit": true,
  "_load_in_8bit": false,
  "bnb_4bit_compute_dtype": "float32",
  "bnb_4bit_quant_storage": "uint8",
  "bnb_4bit_quant_type": "nf4",
  "bnb_4bit_use_double_quant": true,
  "llm_int8_enable_fp32_cpu_offload": false,
  "llm_int8_has_fp16_weight": false,
  "llm_int8_skip_modules": null,
  "llm_int8_threshold": 6.0,
  "load_in_4bit": true,
  "load_in_8bit": false,
  "quant_method": "bitsandbytes"
}

import torch
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

[transformers] `torch_dtype` is deprecated! Use `dtype` instead!

{"model_id": "c0c978bf487e44948717e7e74a819c70", "version_major": 2, "version_minor": 0}

```

1. Prepare Model for K-bit Training
 - PEFT = Parameter Efficient Fine Tuning
 - PEFT is a collection of methods and open-source library designed to adapt large base model (pre-trained models) - like large language model (LLM) to specific downstream tasks by training only a tiny fraction of their parameters.

```

from peft import prepare_model_for_kbit_training

model = prepare_model_for_kbit_training(model)

print("Model prepared for K-bit training")

Model prepared for K-bit training

```

1. Configure LORA using PEFT

```

from peft import LoraConfig

lora_config = LoraConfig(
    r = 16,                                     # r = LoRA Rank , LoRA
    Attention dimension                          # alpha = 32, parameter
    lora_alpha = 32,
    for LoRA Scaling
    lora_dropout = 0.05,                       # dropout : probability
    for LoRA layers
    bias = "none",
    task_type = "CAUSAL_LM",
    target_modules = ["q_proj", "v_proj"]
)

print("LORA config:", lora_config)

LORA config: LoraConfig(task_type='CAUSAL_LM',
peft_type=<PeftType.LORA: 'LORA'>, auto_mapping=None,
peft_version='0.21.2', base_model_name_or_path=None, revision=None,
inference_mode=False, r=16, target_modules={'q_proj', 'v_proj'},
exclude_modules=None, lora_alpha=32, lora_dropout=0.05,
fan_in_fan_out=False, bias='none', use_rslora=False,
modules_to_save=None, init_lora_weights=True,
layers_to_transform=None, layers_pattern=None, rank_pattern={},
alpha_pattern={}, megatron_config=None, megatron_core='megatron.core',
trainable_token_indices=None, loftq_config={}, eva_config=None,
corda_config=None, lora_ga_config=None, use_dora=False,
velora_config=None, alora_invocation_tokens=None, use_qalora=False,
qalora_group_size=16, montecolora_config=None, layer_replication=None,
runtime_config=LoraRuntimeConfig(ephemeral_gpu_offload=False),
lora_bias=False, target_parameters=None, use_bdlora=None,
arrow_config=None, kasa_config=None, ensure_weight_tying=False)

%pip install -U "torchao>=0.16.0"

Collecting torchao>=0.16.0
  Downloading torchao-0.18.0-cp310-abi3-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (21 kB)
  Downloading torchao-0.18.0-cp310-abi3-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (3.4 MB)
  0.0/3.4 MB ? eta -:-:-:-
  1.8/3.4 MB 48.9 MB/s eta
0:00:01 3.4/3.4 MB 58.8 MB/s
eta 0:00:01 3.4/3.4 MB 41.0
MB/s eta 0:00:00
pting uninstall: torchao
  Found existing installation: torchao 0.10.0
  Uninstalling torchao-0.10.0:
    Successfully uninstalled torchao-0.10.0
Successfully installed torchao-0.18.0

```

```

from peft import get_peft_model

model = get_peft_model(
    model,
    lora_config
)

model.print_trainable_parameters()

trainable params: 2,252,800 || all params: 1,102,301,184 || trainable
%: 0.2044

```

1. SFTTrainer
 - Supervised Fine Tuning
 - TRL provides SFTTrainer specially for supervised fine tuning and supports PEFT adapters.

```

from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir = "./tinyllama2-lora",
    num_train_epochs = 1,
    per_device_train_batch_size = 4,
    gradient_accumulation_steps = 1,
    logging_steps = 10,
    save_strategy = "epoch",
    fp16=True,
    bf16=False
)

from trl import SFTTrainer

trainer = SFTTrainer(
    model = model,
    train_dataset = train_dataset,
    eval_dataset = test_dataset,
    processing_class = tokenizer,
    args = training_args
)

print("Trainer Created")

Trainer Created

import inspect

print(inspect.signature(SFTTrainer))

(model: 'str | PreTrainedModel | PeftModel', args:
trl.trainer.sft_config.SFTConfig |
transformers.training_args.TrainingArguments | None = None,
data_collator: collections.abc.Callable[[list[typing.Any]], dict[str,

```

```

typing.Any]] | None = None, train_dataset:
datasets.arrow_dataset.Dataset |
datasets.iterable_dataset.IterableDataset | None = None, eval_dataset:
datasets.arrow_dataset.Dataset |
datasets.iterable_dataset.IterableDataset |
datasets.dataset_dict.DatasetDict |
datasets.dataset_dict.IterableDatasetDict | dict[str,
datasets.arrow_dataset.Dataset |
datasets.iterable_dataset.IterableDataset] | None = None,
processing_class:
transformers.tokenization_utils_base.PreTrainedTokenizerBase |
transformers.processing_utils.ProcessorMixin | None = None,
compute_loss_func: collections.abc.Callable | None = None,
compute_metrics:
collections.abc.Callable[[transformers.trainer_utils.EvalPrediction],
dict] | None = None, callbacks:
list[transformers.trainer_callback.TrainerCallback] | None = None,
optimizers: tuple[torch.optim.optimizer.Optimizer | None,
torch.optim.lr_scheduler.LambdaLR | None] = (None, None),
optimizer_cls_and_kwargs: tuple[type[torch.optim.optimizer.Optimizer],
dict[str, typing.Any]] | None = None, preprocess_logits_for_metrics:
collections.abc.Callable[[torch.Tensor, torch.Tensor], torch.Tensor] |
None = None, quantization_config: 'BitsAndBytesConfig | None' = None,
peft_config: 'PeftConfig | None' = None, formatting_func:
collections.abc.Callable[[dict], str] | None = None)

```

```
inspect.signature(SFTTrainer)
```

```

<Signature (model: 'str | PreTrainedModel | PeftModel', args:
trl.trainer.sft_config.SFTConfig |
transformers.training_args.TrainingArguments | None = None,
data_collator: collections.abc.Callable[[list[typing.Any]], dict[str,
typing.Any]] | None = None, train_dataset:
datasets.arrow_dataset.Dataset |
datasets.iterable_dataset.IterableDataset | None = None, eval_dataset:
datasets.arrow_dataset.Dataset |
datasets.iterable_dataset.IterableDataset |
datasets.dataset_dict.DatasetDict |
datasets.dataset_dict.IterableDatasetDict | dict[str,
datasets.arrow_dataset.Dataset |
datasets.iterable_dataset.IterableDataset] | None = None,
processing_class:
transformers.tokenization_utils_base.PreTrainedTokenizerBase |
transformers.processing_utils.ProcessorMixin | None = None,
compute_loss_func: collections.abc.Callable | None = None,
compute_metrics:
collections.abc.Callable[[transformers.trainer_utils.EvalPrediction],
dict] | None = None, callbacks:
list[transformers.trainer_callback.TrainerCallback] | None = None,
optimizers: tuple[torch.optim.optimizer.Optimizer | None,

```

```
torch.optim.lr_scheduler.LambdaLR | None] = (None, None),
optimizer_cls_and_kwargs: tuple[type[torch.optim.optimizer.Optimizer],
dict[str, typing.Any]] | None = None, preprocess_logits_for_metrics:
collections.abc.Callable[[torch.Tensor, torch.Tensor], torch.Tensor] |
None = None, quantization_config: 'BitsAndBytesConfig | None' = None,
peft_config: 'PeftConfig | None' = None, formatting_func:
collections.abc.Callable[[dict], str] | None = None)>
```

```
trainer.train()
```

```
<IPython.core.display.HTML object>
```

```
TrainOutput(global_step=23, training_loss=2.77411295020062,
metrics={'train_runtime': 9.824, 'train_samples_per_second': 9.161,
'train_steps_per_second': 2.341, 'total_flos': 38853812035584.0,
'train_loss': 2.77411295020062, 'entropy': 2.1693766911824546,
'mean_token_accuracy': 0.44794954856236774, 'num_tokens': 5710.0,
'epoch': 1.0})
```

1. Save The Adapter

```
output_dir = "./llama2-guanaco-lora"
trainer.save_model(output_dir)
tokenizer.save_pretrained(output_dir)
print("Adapter saved: ", output_dir)
```

```
Adapter saved: ./llama2-guanaco-lora
```

```
from peft import PeftModel
```

```
# loaded weights after tuning & updated based model
```

```
base_model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config = bnb_config,
    device_map = "auto"
)
```

```
fine_tuned_model = PeftModel.from_pretrained(
    base_model,
    output_dir,
)
```

```
print("Fine Tuned Adapter Loaded")
```

```
{"model_id": "58a302834d7f499aa0ce1cdc8ab6d699", "version_major": 2, "version_minor": 0}
```

```
Fine Tuned Adapter Loaded
```

```
from transformers import pipeline
```

```
pipe = pipeline(
```

```

    "text-generation",
    model = fine_tuned_model,
    tokenizer = tokenizer,
    max_new_tokens = 200
)

```

[transformers] Passing `generation\_config` together with generation-related arguments=({'max_new_tokens'}) is deprecated and will be removed in future versions. Please pass either a `generation\_config` object OR all generation parameters explicitly, but not both.

1. Running Queries

```

prompt = """
### Human:
How Can I open A bank Account.

```

```

### Assistant:
"""

```

```

result = pipe(prompt)

```

```

print(result[0]['generated_text'])

```

[transformers] Both `max\_new\_tokens` (=200) and `max\_length` (=2048) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer LlamaTokenizer. The clean_up_tokenization post-processing step is designed for WordPiece tokenizers and is destructive for BPE (it strips spaces before punctuation). Set clean_up_tokenization_spaces=False to suppress this warning, or set clean_up_tokenization_spaces_for_bpe_even_though_it_will_corrupt_output=True to force cleanup anyway.

```

### Human:
How Can I open A bank Account.

```

```

### Assistant:
Bank accounts are opened through the bank's branch. To open a bank account, you need to provide certain documents such as identity proofs, proof of address, and proof of income. Here are the steps to open a bank account:

```

1. Go to the bank's branch.
2. Provide identity proofs such as a national identity card or passport, driving license, and voter id.
3. Provide proof of address such as a utility bill, rental agreement,

- or property deed.
4. Provide proof of income such as a pay slip or salary cheque.
 5. Pay the bank's opening fee.
 6. Receive the first bank statement.
 7. Sign the required documents and pay the bank the account opening fee.
 8. If you're a new customer, the bank will set up your account.
 9. Receive your account number, which you can use

```
prompt = ""
```

```
### Human:
```

```
My Credit Card Is Lost What Should I do Now?.
```

```
### Assistant:
```

```
""
```

```
result = pipe(prompt)
```

```
print(result[0]['generated_text'])
```

[transformers] Both `max\_new\_tokens` (=200) and `max\_length` (=2048) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

```
### Human:
```

```
My Credit Card Is Lost What Should I do Now?.
```

```
### Assistant:
```

If you lost your credit card, the next step is to file a police report with the local police station and provide a copy of your card with the name and address of the card holder. Once you have this information, you can report the loss to your bank, which will initiate a lost card notice. This will provide you with an alert to take immediate steps to prevent any unauthorized transactions on your card. It's best to contact your bank immediately after you have lost your credit card. They will assist you in the process of replacing your card. To replace the lost card, you will need to provide the bank with the new card information, including the card number, expiry date, and cardholder's name. The bank will then send you a new card, which you can use in the interim. It is best to replace your card as soon as possible to avoid any inconvenience or fraudulent activity on your account.

```
prompt = ""
```

```
### Human:
```

```
I want To avail Net Banking Facility.
```

```

### Assistant:
"""

result = pipe(prompt)

print(result[0]['generated_text'])

[transformers] Both `max_new_tokens` (=200) and `max_length` (=2048)
seem to have been set. `max_new_tokens` will take precedence. Please
refer to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main\_classes/text\_ge
neration)

### Human:
I want To avail Net Banking Facility.

### Assistant:
There are several ways to avail Net Banking facility. Some of them
are:
1. Online Net Banking: You can log into your bank's online banking
website using your unique login credentials.
2. Mobile Banking: You can use your mobile phone number and 4-digit
PIN to login to your bank's mobile banking app.
3. Bank Branch: You can also opt for the traditional branchless
banking, where you can avail services such as deposits, withdrawals,
and transfers.
4. ATMs: Some banks have ATMs that offer a wide range of services such
as deposits, withdrawals, and transfers.

### Human:
Can you tell me more about mobile banking?

```

Banking FAQ Chatbot: Architecture, Design Choices, and Test Results

1. Project Overview

The objective of this project is to fine-tune a language model to answer customer queries related to banking services. The system uses a curated dataset of 100 frequently asked questions (FAQs), covering account management, debit cards, digital banking, fund transfers, loans, deposits, and banking security. The goal is to develop a customer-support assistant that generates relevant, clear, and consistent responses to common banking questions.

2. System Architecture

The project follows a dataset-driven supervised fine-tuning architecture consisting of the following components:

1. **Data Collection:** Banking FAQs are stored in a JSON file. Each record contains two fields: `question` and `response`. The dataset is loaded into Google Colab using the Hugging Face `datasets` library.
2. **Data Preprocessing:** The question and corresponding response are combined into a single `text` field using a consistent instruction format. This prepares the dataset for supervised fine-tuning (SFT).
3. **Base Model and Parameter-Efficient Fine-Tuning:** A pretrained language model is used as the foundation. The PEFT library and Low-Rank Adaptation (LoRA) configuration are intended to adapt the model to banking customer-support tasks while reducing the number of trainable parameters.
4. **Training Pipeline:** The `TRL SFTTrainer` is used to manage supervised fine-tuning. PyTorch provides the underlying tensor operations, while Hugging Face Transformers manages model loading and tokenization. Google Colab supplies the GPU-based execution environment.
5. **Evaluation and Inference:** After successful training, the model should be evaluated on unseen banking questions. Its generated responses can then be assessed for relevance, correctness, clarity, and safety.

Architecture flow:

Banking FAQ JSON → Dataset Loading → Text Preprocessing → Pretrained Model + LoRA → SFT Training → Evaluation → Banking FAQ Responses

3. Design Choices

Dataset Format

JSON was selected because it is lightweight, easy to maintain, and compatible with common machine-learning data pipelines. The dataset contains 100 question–response pairs. The two-column structure preserves the relationship between each customer query and its expected answer.

Supervised Fine-Tuning

Supervised fine-tuning was selected because the task involves learning to respond to customer questions using example answers. Formatting each record consistently as a question and an answer helps establish the intended interaction pattern.

LoRA and PEFT

LoRA-based parameter-efficient fine-tuning was chosen to reduce training requirements compared with updating every parameter in the base model. It is particularly useful in resource-constrained environments such as Google Colab. The final configuration must remain compatible with the selected base model, installed libraries, and available GPU.

Hardware and Precision

The development environment uses an NVIDIA Tesla T4 GPU with compute capability 7.5. FP16 is a potential training precision option for this GPU. Training precision and quantization settings must be configured consistently to avoid unsupported operations and excessive memory use.

Banking Safety

Banking responses should be factual, understandable, and cautious. The assistant should not request passwords, PINs, CVVs, or OTPs, and should direct users to official bank channels for account-specific matters. The dataset is a small prototype and should not be treated as a source of authoritative financial advice.

4. Test Results

The following tests have been performed or identified during development:

Test	Result
JSON dataset loading	Successful; 100 FAQ records are available in the dataset
Dataset schema validation	Successful after formatting records into a <code>text</code> field
Trainer dataset compatibility	The missing <code>text</code> column issue was addressed through preprocessing
GPU detection	Successful; NVIDIA Tesla T4 detected
Mixed-precision compatibility	An unsupported BF16 operation was encountered; FP16 configuration is the proposed correction
LoRA model preparation	Blocked by a TorchAO/PEFT version incompatibility
End-to-end fine-tuning	Not yet verified as successful
Generated-response quality	Not yet evaluated on a held-out test set

The implementation has reached the model preparation and training setup stages, but successful end-to-end fine-tuning and response-quality evaluation remain pending. The TorchAO dependency conflict and mixed-precision configuration must be resolved before final training results can be reported.

5. Conclusion and Future Improvements

This project establishes a practical pipeline for preparing banking FAQ data for language-model fine-tuning. The architecture combines structured JSON data, Hugging Face Datasets, Transformers, TRL, PEFT, and LoRA within Google Colab.

The next steps are to establish compatible package versions, complete model training, and evaluate the resulting model on questions that were not included in the training dataset. Future improvements should include expanding the FAQ dataset, adding diverse customer phrasings, measuring answer quality and hallucination rates, and testing responses to sensitive or ambiguous banking requests.

Overall, the project provides a foundation for a domain-specific banking customer-support assistant. Its effectiveness will depend on successful training, systematic evaluation, and appropriate safeguards before real-world deployment.